

Digital Signal Processing

2nd Exercise

Full Name: Katsaidonis Nikolaos

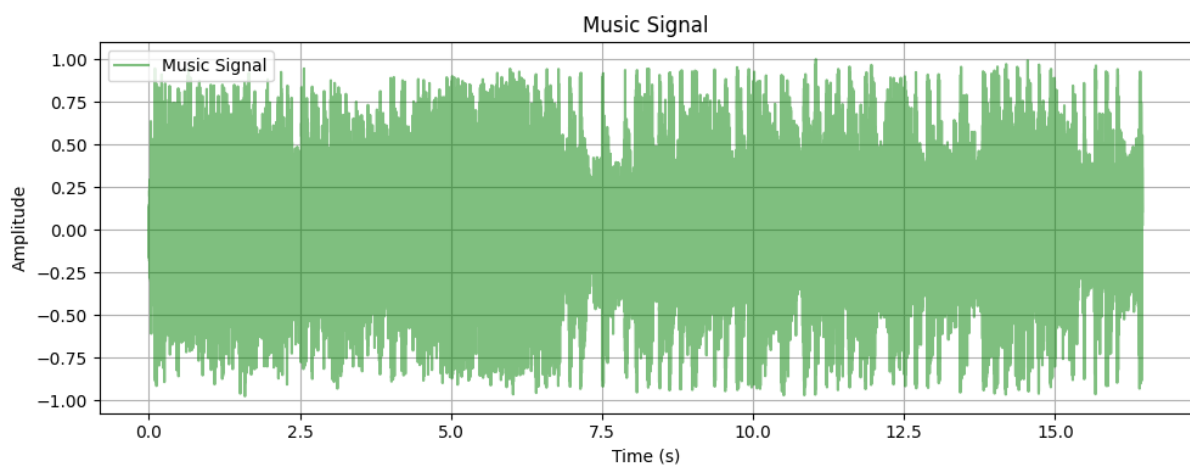
Reg. Number 03121868

Part 1. Psychoacoustic Model 1

Modern sound coding models, in order to compress a recording, work by **emphasizing the frequencies perceived** by the **human hearing system** according to the psychoacoustic model. To achieve the encoding of music signals we used algorithms such as MPEG, which is based on spectral analysis, detection **of masks of tones and noise and calculation of coverage thresholds**.

Step 1.0: Pre-process the signal

In this first step we read the **music.wav** signal where we converted it from stereo format to **mono**. We then normalized this signal along its entire length in the interval $[-1,1]$, dividing its samples by its absolute maximum value. This is how we navigated the music signal as shown below:

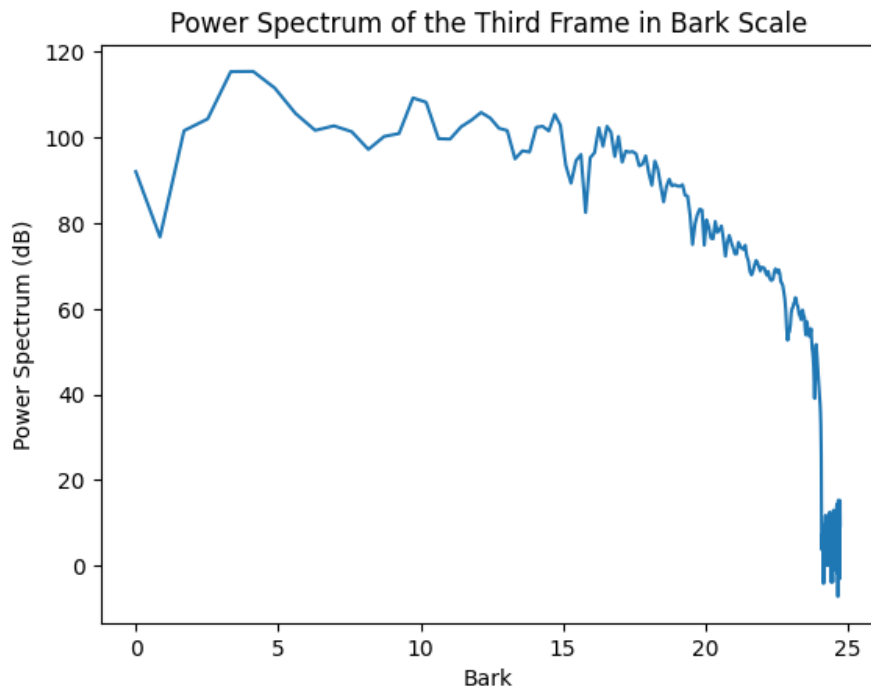


Also at this point **we window the signal with Hanning windows** of size 512 samples to then analyze frames.

Step 1.1: Spectral Analysis

In this step we wanted to express the signal spectrum in SPL units that express the **air pressure of the eardrum**. To achieve this, we define the **Bark scale** and calculate the **power spectrum $P(k)$** of the signal with $N = 512$ samples according to the MPEG Layer-1 standard. Because the power spectrum $P(k)$ has **symmetry equivalent to the measure of DFT**, finally we keep the **one-tailed power range**, corresponding to $k \in [0, N/2]$ according to the given formula with **PN=90.302dB**:

$$P(k) = PN + 10 \log_{10} \left| \sum_{n=0}^{N-1} w(n)x(n)e^{-j\frac{2\pi kn}{N}} \right|^2, 0 \leq k \leq \frac{N}{2}.$$

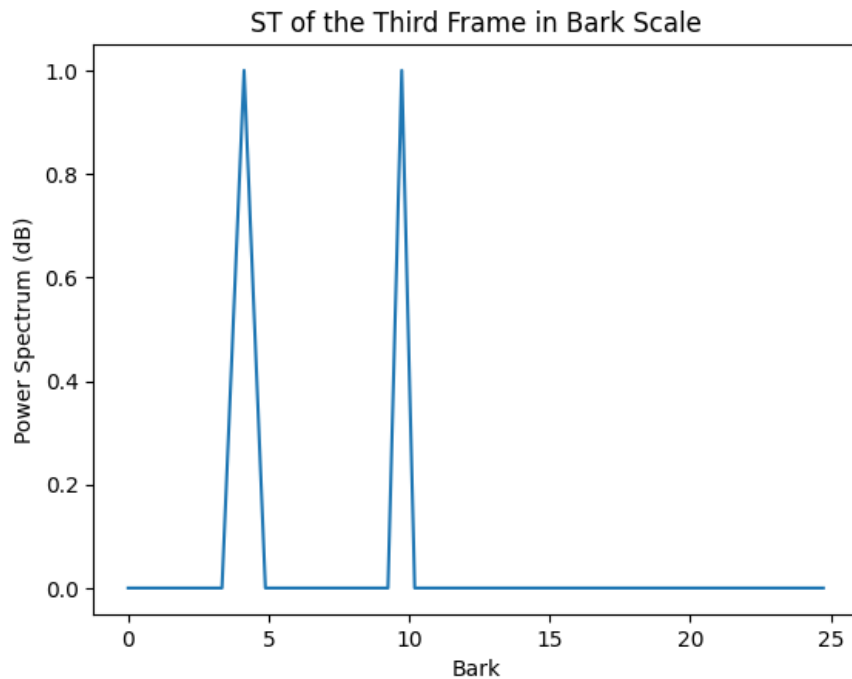


We chose to work with the **third window** for which we navigated the power spectrum on a bark scale, as shown above. We observe that the power spectrum of the windowed signal in **the low and medium frequencies is relatively consistently high** in contrast to the high frequencies where it tends to 0.

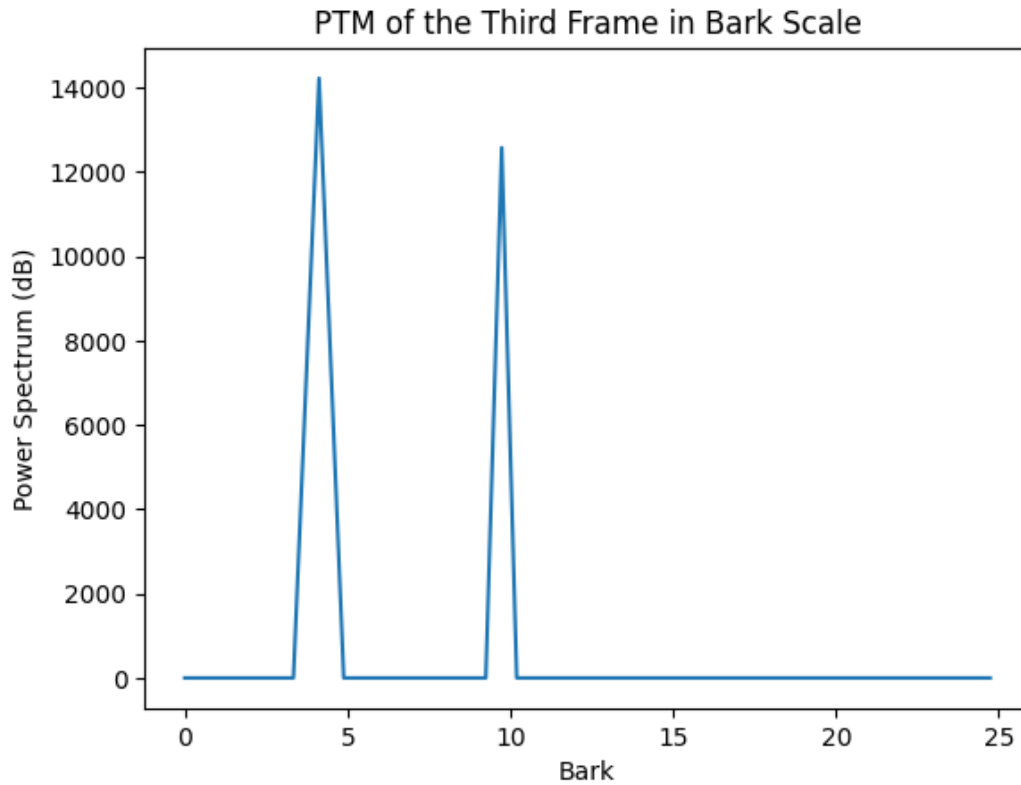
Step 1.2: Detect tone and noise masks (Maskers)

After calculating the power spectrum, we want to identify the local maxima (masks) per critical band. Local maxima are the largest power spectrums compared to their neighboring frequencies by at least 7db. The **ST(k)** function listed below **returns 1 if there is a tonal mask at k, and 0 else:**

$$S_T(k) = \begin{cases} 0, & \forall k \notin [2, 250) \\ P(k) > P(k \pm 1) \wedge P(k) > P(k \pm \Delta_k) + 7\text{dB}, & \forall k \in [2, 250) \end{cases}$$



Below we see the strength of PTM(k) mask tones for the third frame we chose:

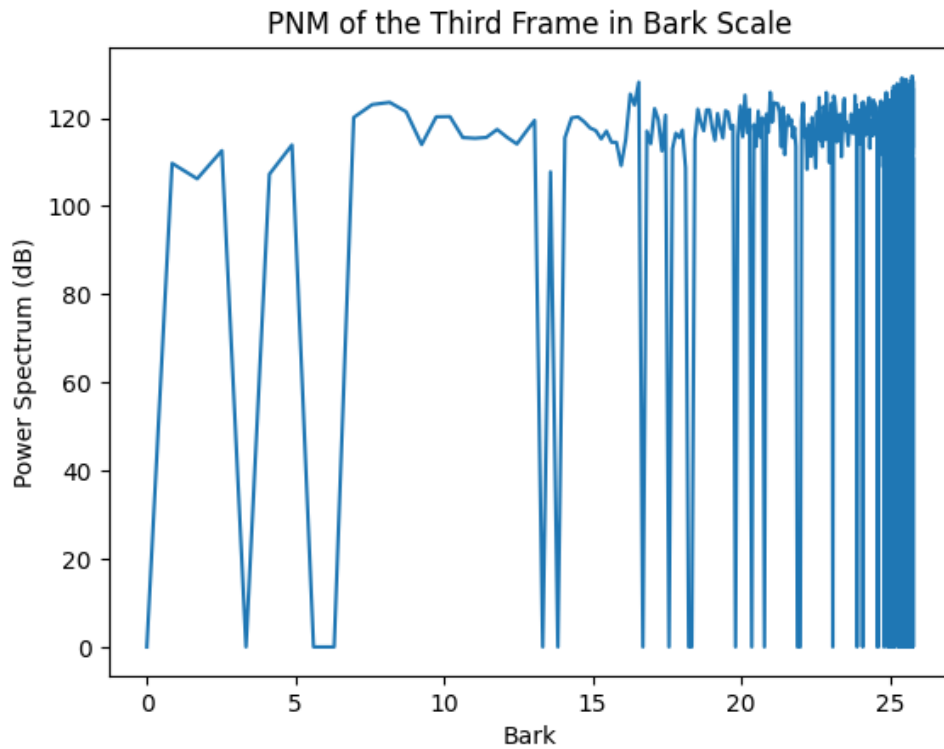


Power is given by the following formula:

$$P_{TM}(k) = \begin{cases} 10 \log_{10}(10^{0.1(P(k-1))} + 10^{0.1(P(k))} + 10^{0.1(P(k+1))}) (\text{dB}), & \alpha \vee S_T(k) = 1 \\ 0, & \alpha \vee S_T(k) = 0 \end{cases}$$

What we observe is that the **power is zero for the most part, except for the points where we have a mask**, which stand out against their neighbors.

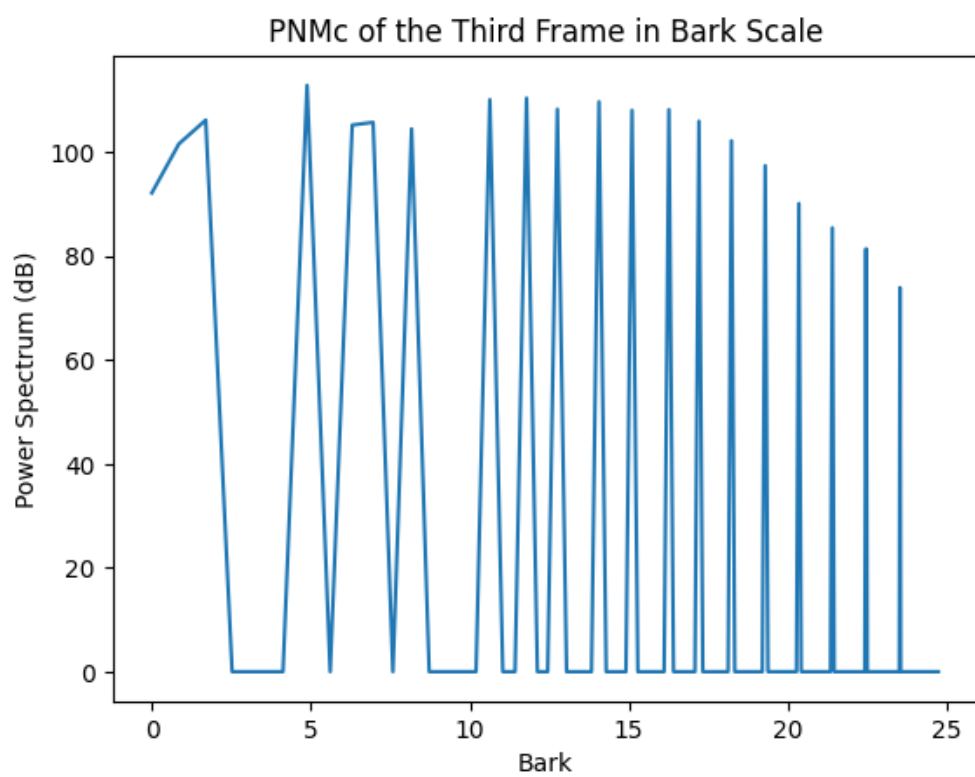
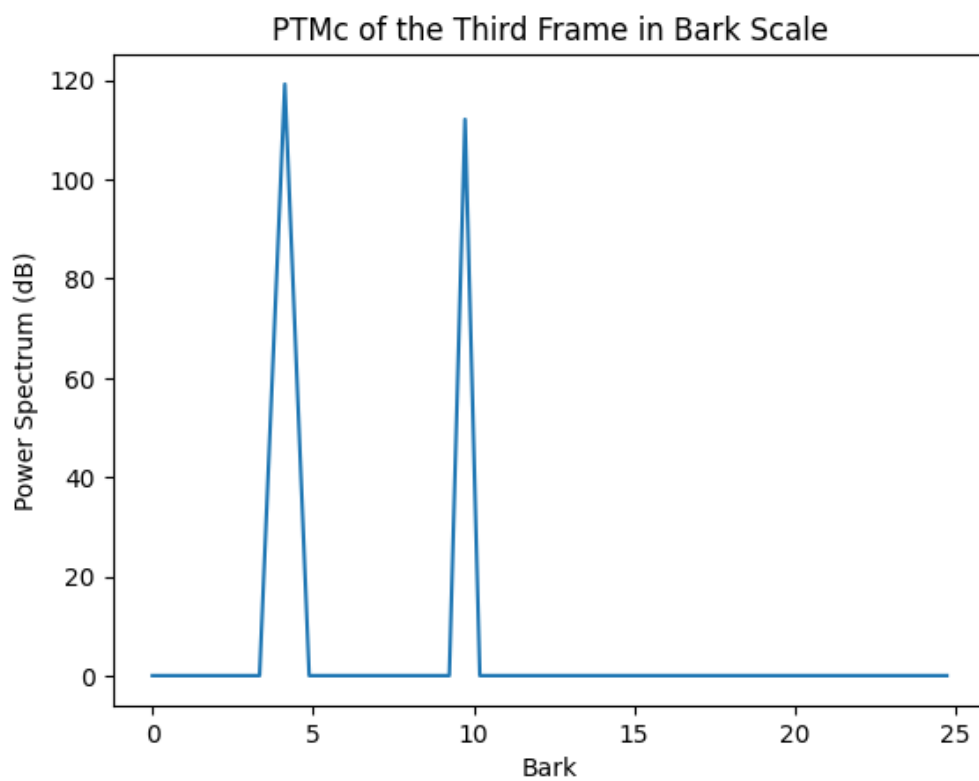
Finally, in this step we will load the P_NM table that contains noise masks for each window of the signal which we represent for the frame we have chosen:



Step 1.3: Reduce and reorganize masks

In this step we will reduce and reorganize masks. More specifically, every tone mask and every noise mask that is **below the absolute hearing threshold is discarded**, while in addition, in the windows of 0.5 Bark we find the masks and **replace them with the loudest mask**.

After loading the tables of reduced and reorganized masks P_{TMc} , we P_{NMc} quote the graphs for the selected frame:



Step 1.4: Calculate the two different Individual Masking Thresholds

According to the following formulae we calculate the **coverage thresholds** for the two types of masks which represent the percentage of coverage at each point i , derived from a mask at point j :

$$T_{TM}(i,j) = P_{TM}(j) - 0.275b(j) + SF(i,j) - 6.025(\text{dB SPL}),$$

$$T_{NM}(i,j) = P_{NM}(j) - 0.175b(j) + SF(i,j) - 2.025(\text{dB SPL})$$

The displayed $SF(i,j)$ function indicates the **minimum power level needed by adjacent frequencies to be perceived by the human auditory system** and is given by:

$$SF(i,j) = \begin{cases} 17\Delta_b - 0.4P_{TM}(j) + 11, & -3 \leq \Delta_b < -1 \\ (0.4P_{TM}(j) + 6)\Delta_b, & -1 \leq \Delta_b < 0 \\ -17\Delta_b, & 0 \leq \Delta_b < 1 \\ (0.15P_{TM}(j) - 17)\Delta_b - 0.15P_{TM}(j), & 1 \leq \Delta_b < 8 \end{cases}$$

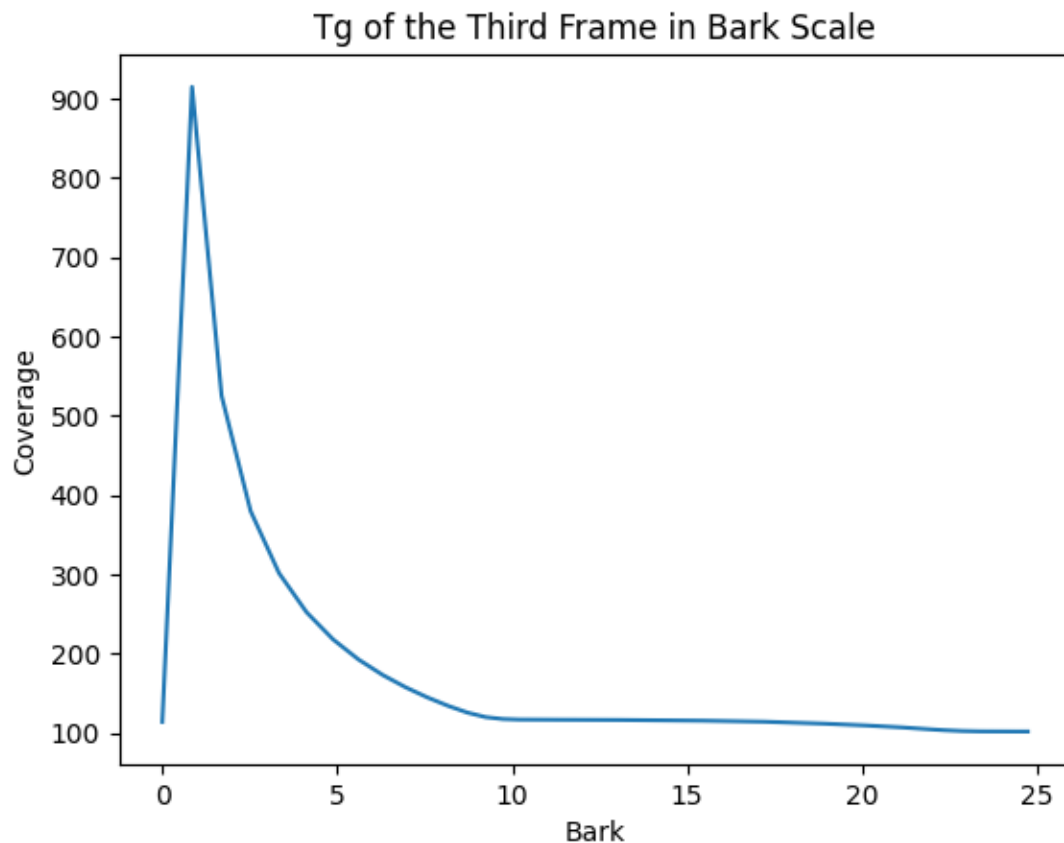
In this model we assumed that the threshold TTM is defined in a neighborhood interval of the 12-Bark of the mask at point j , i.e. at positions $i : b(i) \in [b(j) - 3, b(j) + 8]$.

Step 1.5: Calculate the Global Masking Threshold

From the two thresholds we calculated, the total for each frame can now be obtained according to the formula:

$$T_g(i) = 10 \log_{10} \left(10^{0.1T_q(i)} + \sum_{l=0}^{255} 10^{0.1T_{TM}(i,\ell)} + \sum_{m=0}^{255} 10^{0.1T_{NM}(i,m)} \right) \text{ dB SPL},$$

Below is the threshold graph for the 3rd frame on a Barkscale scale:

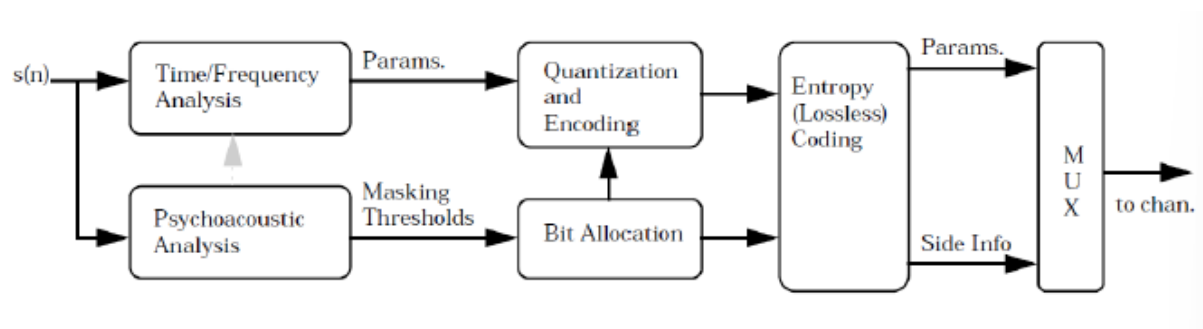


We observe that for **high frequencies the threshold is zero**, while for lower frequencies it **displays a peak that descends towards the low**. This means that at high frequencies there is no coverage from adjacent frequencies. Thus, in the human auditory system, **low frequencies are the ones that overlap most easily, resulting in difficulty in distinguishing them**.

Part 2. Time-Frequency Analysis with Zone Pass Filter Array

The **purpose** of the exercise is to familiarize students with concepts such as **quantization** and **coding** of a sound signal. For this purpose, some **band-pass filter arrays** will be used which divide the spectrum into frequency subbands. The usefulness of these is shown in the identification of perceptually unnecessary points of the sound signal that they provide us.

More specifically, the process of the following scheme will be implemented:



Step 2.0: Filter Filter Array

Per time frame, we use band-pass filter arrays designed based on **discrete cosine transformation** (MDCT). We used **M=32** analysis and composition filters, **hk[n]** and **gk[n]** respectively, whose shock responses are shown below:

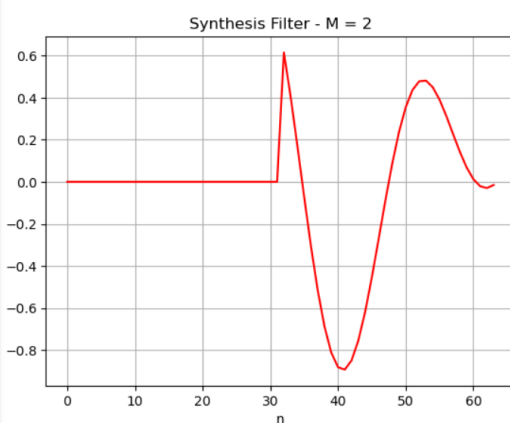
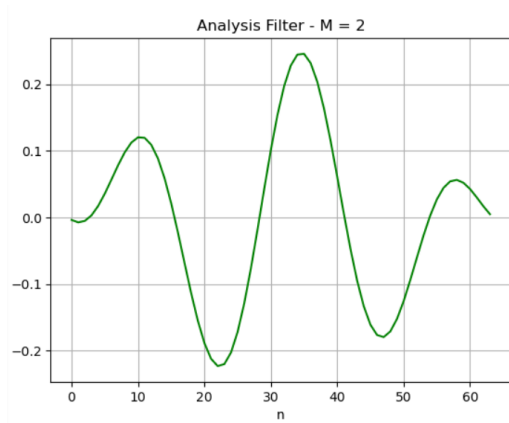
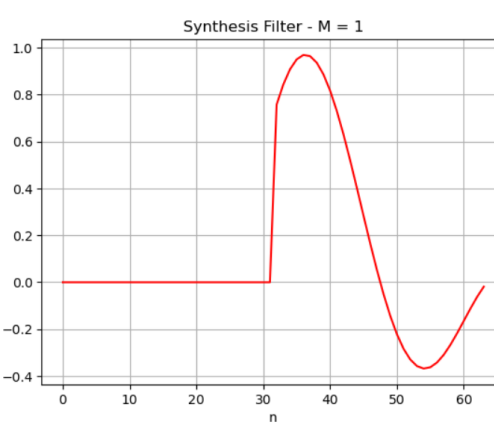
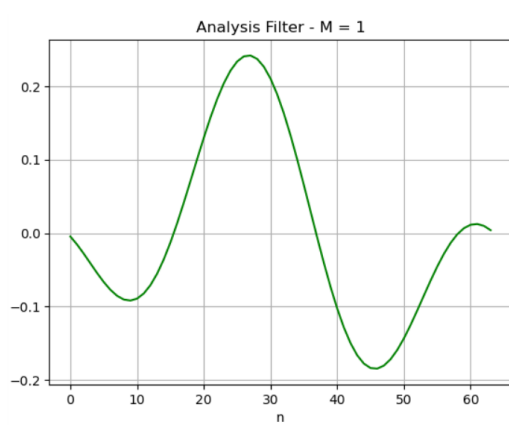
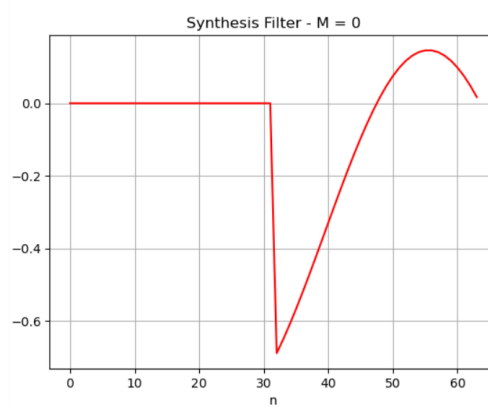
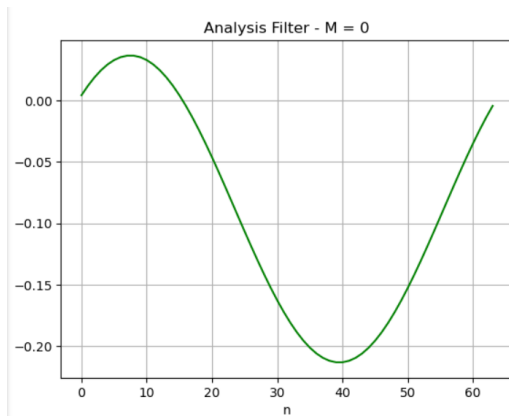
$$h_k(n) = \sin \left[\left(n + \frac{1}{2} \right) \frac{\pi}{2M} \right] \sqrt{\frac{2}{M}} \cos \left[\frac{(2n + M + 1)(2k + 1)\pi}{4M} \right]$$

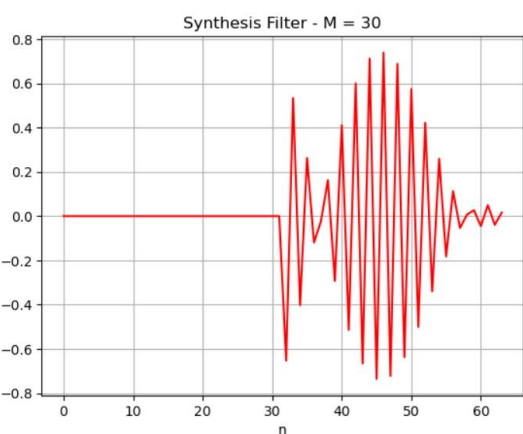
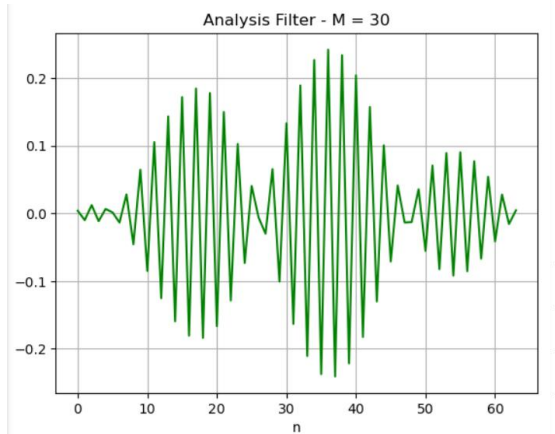
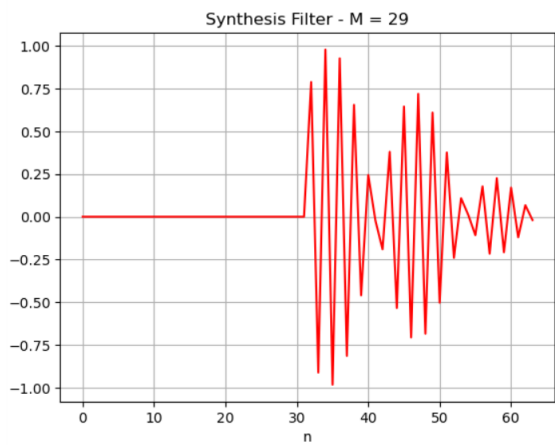
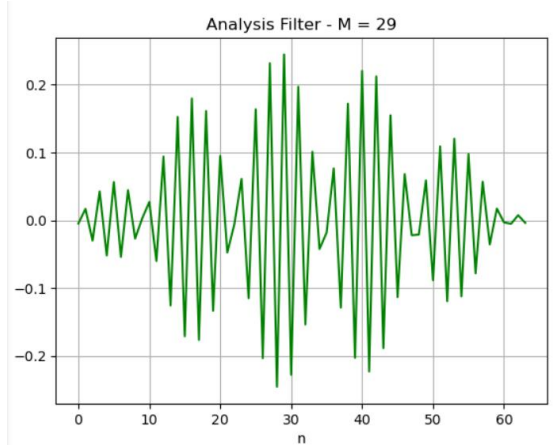
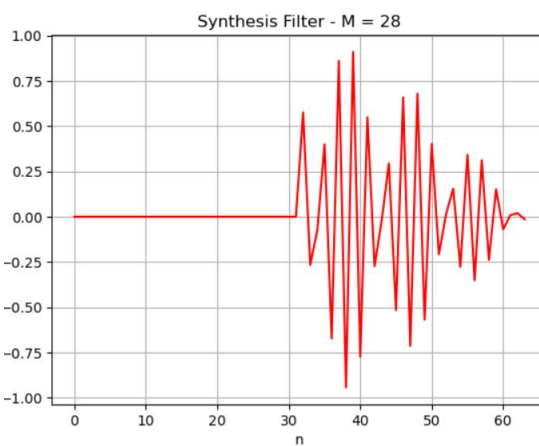
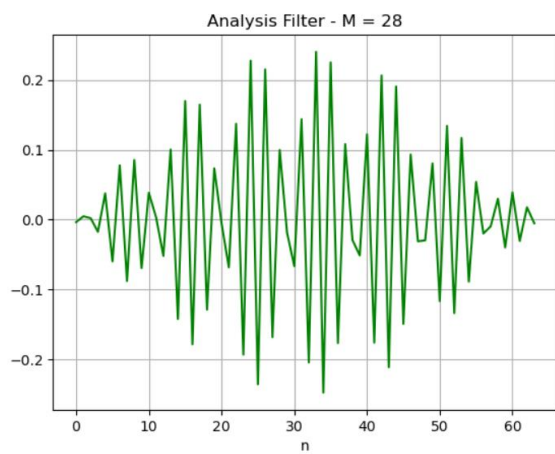
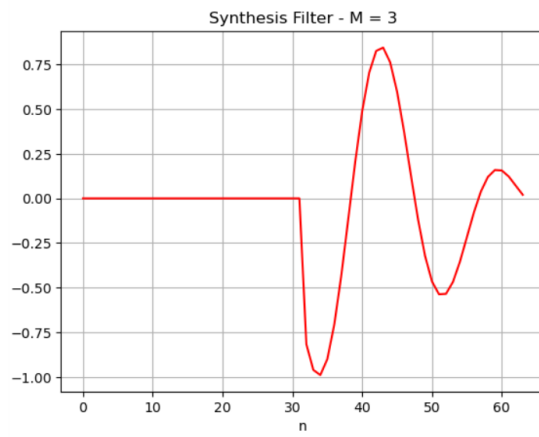
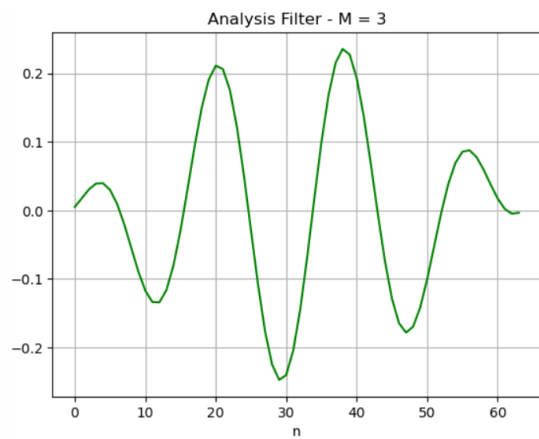
$$g_k(n) = h_k(2M - 1 - n)$$

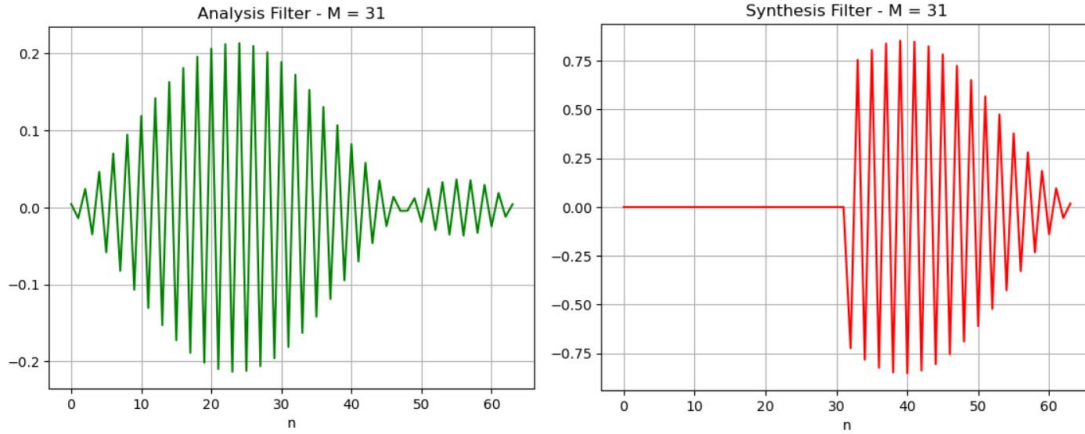
Note that these filters are of length $L=2M$ with $0 \leq n \leq L - 1$ and $0 \leq k \leq M - 1$.

The construction code of the filters is shown in the corresponding notebook file.

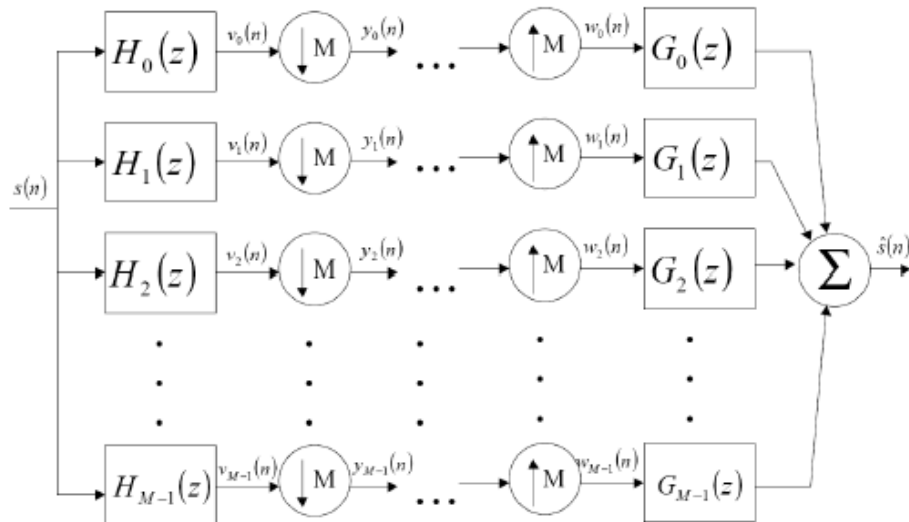
To get an idea of the impact responses of the filters, plot the first four and the last four analysis and composition filters (for $M=0.1, \dots, 3$ and $M=28, 29, \dots, 31$) which we quote immediately below:







We schematically present the procedure that will follow in the following questions:



Step 2.1: Analyze with a Filter Array

In the corresponding file, code has been created with which **we convolve $x[n]$ with the $h_k[n]$ analysis filters**, that is, we pass the signal through the filters and get outputs equal to the convolutions of the signals with the shock response of each of the filters. Below is the mathematical calculation of convolutions **$u_k[n]$** :

$$v_k(n) = h_k(n) * x(n) = \sum_{m=0}^{L-1} x(n-m)h_k(m), \quad k = 0, 1, \dots, M-1$$

This is accomplished by using the **convolve()** function in the numpy library.

This is followed by a **subsampling of** each $u_k[n]$ **by factor $M=32$** so that the original signal is divided into the time components $y_k(n) = u_k(Mn)$. This is achieved by the python slicing method ($y = u[:,M]$).

It is worth noting that we consider the coatings introduced by non-ideal filters to be negligible.

Step 2.2: Quantization

The function of the **quantizer** involves assigning the values of samples of a discrete sequence to **numbered levels of quantization** (levels). We will construct two kinds of quantizers, an **adaptive uniform quantizer of 2^{B_k} planes**, where **B_k** is the number of encoding bits of the sequence $y_k(n)$ in the current frame $x[n]$, and an **unadaptive quantizer** with constant **$B_k=8$ bits**. B_k is calculated through the formula:

$$B_k = \text{int} \left(\log_2 \left(\frac{R}{\min(T_g(i))} \right) - 1 \right)$$

The quantization pitch D is adjusted based on B_k and the value range $[x_{\min}, x_{\max}]$,

$$\Delta = \frac{x_{\max} - x_{\min}}{2^{B_k}}$$

while **$T_g(i)$** is the total coverage threshold of the psychoacoustic model as calculated in **Part 1** and **$R = 2B$** the number of its intensity gradients

initial signal $s(n)$, encoded in B bits per sample .

Step 2.3: Composition

In this step, the sequences subjected to quantization are **oversampled by factor M**. In fact, we get our quantized signal and every 0,M,2M,... samples **we interpolate zeros** thus creating the signal **wk[n]** (i.e. w is the same as the signal at positions 0,M,2M,... and elsewhere zero). This signal is then passed through the **synthesis filters g_k[n]** that we constructed in a previous question, i.e. it is convoluted, and the output signal **x[^][n]** is obtained. Finally, we apply the **OverLap-Add** method to obtain the music signal **s[^][n]**.

APPLICATION OF THE ABOVE TO THE MUSIC SIGNAL

To implement the above, as shown in the code file, we have also listed some useful functions in part one.

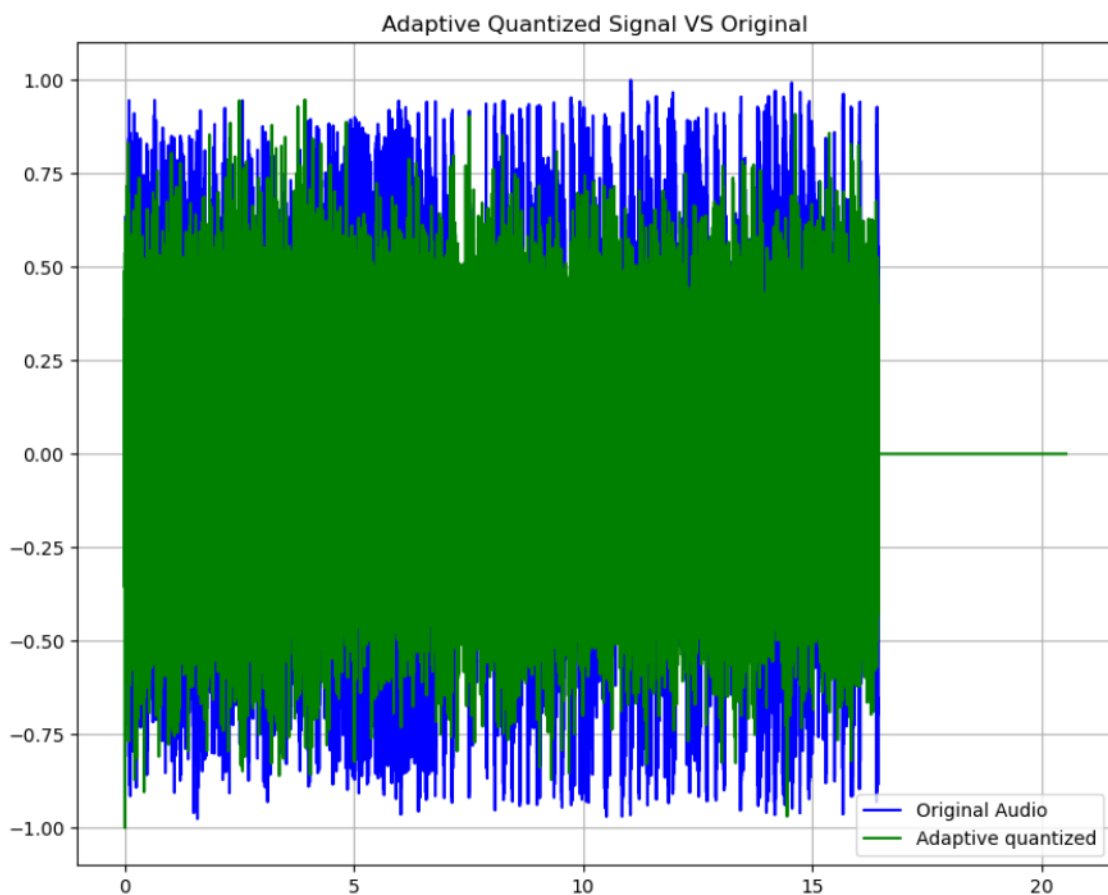
First we load the music signal via **librosa.load()** and normalize to [-1,1]. Then we divide our signal into **N = 512 frames of length 512** through **librosa.util.frame()** and finally we window the frames with **a hanning window** of length N = 512. We printed the first 5 **frames**.

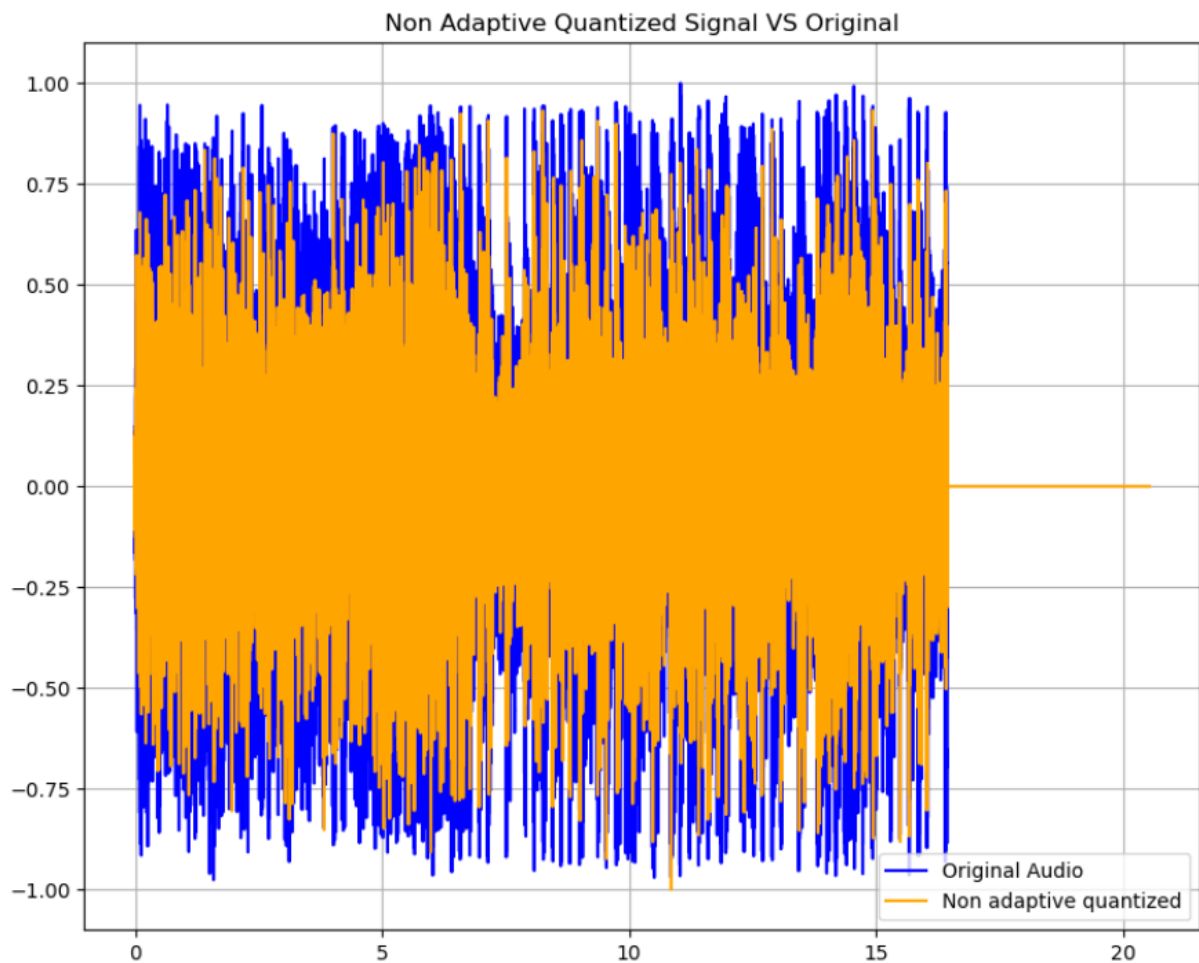
```
Length of each frame=1416
Frame 0:
[-0. -0. -0. ... -0.  0. -0.]
Frame 1:
[-1.21434314e-07 -3.69348030e-06  2.58121242e-07 ... -2.99526235e-05
 1.18536319e-05 -7.80171451e-06]
Frame 2:
[-8.04838535e-07 -1.19857592e-05  9.72141696e-06 ... -1.28816409e-04
 4.19689301e-05 -2.40067157e-05]
Frame 3:
[-2.38620783e-06 -1.90632664e-05  3.44205200e-05 ... -2.85590012e-04
 5.98558068e-05 -4.93501535e-05]
Frame 4:
[-4.99252946e-06 -1.49212817e-05  6.80279048e-05 ... -4.70029737e-04
 7.91766334e-05 -8.74067991e-05]
```

Then, after loading the mask files, the analysis **filtering process follows**. We pass the frames of the signal sequentially through the 32 hk analysis filters and obtain the **signals uk** followed by a **subsampling by factor M=32** (with slicing $v = v[:, :32]$).

Next, we pass the signals through a quantizer which in one case is **adaptive while in the other it is non-adaptive**. This is followed by **oversampling by $M=32$** , i.e. insertion of zeros in the signal. The frames will then go through the **32 gk composition filters** and the individual signals will be added for the final creation of the reconstructed signal.

We list the final reconstructed music signals in relation to the input signal using adaptive and non-adaptive quantizer below:



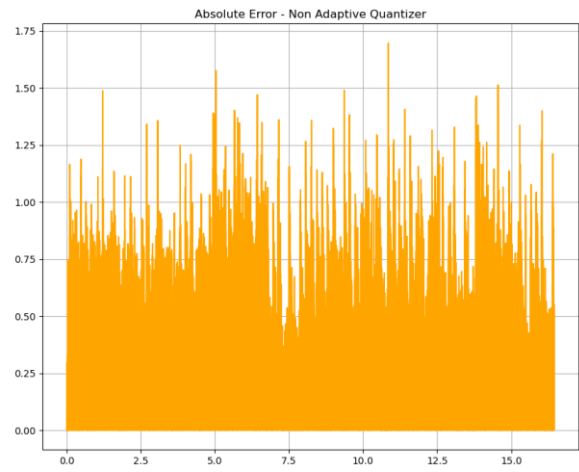
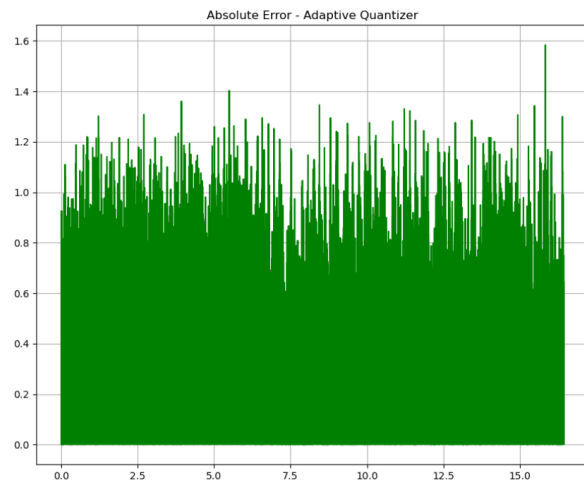


We store the two signals in **.wav** files and notice that the **noise in the adaptive quantized signal is noticeably less** as expected. In addition, the two signals are **1769KB in size compared to 2790KB of the original** which indicates **successful compression of the music_dsp2024 file**.

Finally, we calculate the mean square error for the two quantizers. **We emphasize that due to an error that was not detected, the two quantizers produce almost the same error with the non-adaptive one producing slightly less:**

```
Mean Square Error - Adaptive Quantizer= 0.09412908116619181
Mean Square Error -Non Adaptive Quantizer= 0.09120078425211521
```

Here are the graphs of the absolute values of the errors:



Note that the final number of bits of the adaptive quantizer was calculated equal to 11328 bits.